

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-add372a92aa4d49c2.jsonl`  
- SHA-256: `9f939284f3776d8c0863f43b67f79bbcd9ebe283c88e4ac858716f86bd28d05f`  
- Source modified: `2026-07-06T09:09:16+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:07:16.620Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel
creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

DIMENSION: test coverage + discoverability.

Investigate: the ingestion gate lives inside on_new_message which is '# pragma: no cover' ? so the core gating

behavior (skip unwatched, pass-through/auto-watch new) has NO test. Could the gate be extracted into a testable helper? A test would have caught the ADR default-watched issue. Also: does format_commands() list the new /watch, /unwatch, /channels, /watchlist commands? (I believe NO ? they're undiscoverable via /show commands, and the existing test_format_commands_includes_all_sections doesn't check them.) Do the 10 new tests assert persistence (they seem to ? good) and cover unwatch-unknown, empty states? Any coverage gaps in the new db functions. Report findings.

2. assistant (2026-07-06T09:07:19.453Z)

I'll review PR #39 against ADR 0009. Let me start by reading the ADR spec and the changed files.

3. user (2026-07-06T09:07:19.968Z)

```
1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans **every** channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a *record* of seen channels;
13     it is not used as a filter.
14
15     The product vision is a **multiplexing video re-distribution system**:
16
17     - A small set of high-quality **source** channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed **destination** broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to *outputs* by topic, not to *inputs* by source.
22
23     This requires two capabilities the current architecture does not have:
24
25     1. **Selective scanning** ? choose which source channels are watched, rather
26       than ingesting all of them.
27     2. **Per-source ? per-destination routing** ? a match from source channel X is
28       delivered to a specific destination channel Y, instead of the single global
29       `delivery_target` (ADR 0005).
30
31     ## Decision
32
33     Add three additive, independently-shippable capabilities:
34
35     ### 1. Selective scanning (watched-flag)
36
37     Add a `watched` flag to the `channels` table. The default is locked as
38     **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
39     explicitly `/watch` a channel. This matches the multiplexing product shape
40     (sources are a *curated* set) and avoids blindly re-ingesting every chat the
41     account is a member of.
42
43     Gate `on_new_message` at the top: after computing `channel_id`, look up the
44     row and `return` early if `watched` is false, before any dedup or filter work.
45
46     This is a small migration plus ~5 lines in `telethon_client.py`.
47
48     **Discoverability note.** With `DEFAULT 0` plus the current lazy population
```

```

49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id  INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.
89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.
95 - Each capability is additive and independently deployable; none disturbs the
96   existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98   file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100   `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105   may be a member of a *destination* broadcast channel and re-see its own
106   output, producing an infinite forward loop. The contract is: **a channel
107   is either a source or a destination, never both.** This is enforced in
108   `on_new_message` by excluding any channel that is a configured destination
109   (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110   user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111   this; the invariant is stated here so the handler has a contract to code
112   against.
113 - Per-route filter overrides add config complexity; v1 may ship with global

```

```

114     filters only and a single implicit route (`source ? global target`), then
115     layer explicit per-route rules later.
116
117     ## Backwards compatibility
118
119     Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120     install from receiving matches until the user re-watches their channels ? a
121     behaviour change disguised as a schema change. To preserve the current
122     "ingest everything" semantics for existing installs, the migration backfills:
123
124     ```sql
125     ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126     UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127     ```
128
129     i.e. any channel the install has *already* seen traffic from is implicitly
130     watched, so pre-migration users observe no change. New channels discovered
131     after the migration default to `watched = 0` and must be explicitly watched ?
132     which is the intended new UX. This mirrors the "old rows behave exactly as
133     before" pattern established by ADR 0005.
134
135     ## Rollout order
136
137     1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138       `/channels`). Smallest unit; immediately useful even without routing.
139     2. **Named destination channels** ? extend `/set delivery_target` to allow
140       switching among a user-defined set, not just one global value.
141     3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142       `/route <source> <dest>` command). The full multiplexing vision.
143
144     Each step is shippable and useful on its own; step 3 is not required to
145     deliver value from steps 1?2.
146
147     ## Alternatives considered
148
149     - **Server-side `chats=` filter at handler registration:** rejected ? the
150       watched set is per-user and mutable at runtime; re-registering Telethon
151       handlers on every `/watch` change is brittle. An in-handler lookup against
152       the `channels` table is simpler and hot-swappable.
153     - **Content-based classification (auto-detect genre):** explicitly out of
154       scope for v1. Routing is user-defined by source channel; automatic topic
155       detection could be a future enhancement but is not required to deliver the
156       multiplexing UX.
157
158     ## References
159
160     - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161     - ADR 0005 (per-user delivery target) ? the single-target delivery this
162       generalises into per-route targets.
163     - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164       the original `document_id`.
165     - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166       `/unwatch`, and routing commands to public users.
167

```

4. user (2026-07-06T09:07:22.246Z)

```

db87cc0 feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)
12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
---DIFF---
src/tg_video_filter/command_service.py | 104 ++++++
src/tg_video_filter/commands.py         |   6 +-

```

```

src/tg_video_filter/db.py          | 28 ++++++++
src/tg_video_filter/telethon_client.py | 9 +++
tests/test_command_service.py      | 110 ++++++
5 files changed, 254 insertions(+), 3 deletions(-)

```

5. assistant (2026-07-06T09:07:24.117Z)

Now let me read the four changed source files and the tests.

6. user (2026-07-06T09:07:25.132Z)

```

1      """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3      Extracts command logic from the Telethon adapter (`commands.py`) so the
4      same `/set`, `/show` commands work identically through a Bot API
5      front-door. The service is pure: strings in, `CommandResult` out ? no
6      Telethon objects, no network calls.
7
8      See `docs/adr/0007-inline-config-commands.md` for the command model and
9      `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API motivation.
10     """
11
12     from __future__ import annotations
13
14     import logging
15     import re
16     from dataclasses import dataclass
17
18     import aiosqlite
19
20     from tg_video_filter import db
21     from tg_video_filter.models import FilterConfig, Source
22
23     log = logging.getLogger(__name__)
24
25     # Key into the shared engine_holder dict (imported by commands.py's adapter).
26     ENGINE_KEY = "engine"
27
28     # --- regex patterns -----
29
30     _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmg]?b?)$", re.IGNORECASE)
31     _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
32     _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
33     _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
34 re.IGNORECASE)
35     _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
36     _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
37     _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
38 re.IGNORECASE)
39     _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
40     _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
41
42     # --- source scanning regex (ADR 0009, Track E) -----
43
44     _WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
45     _UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
46     _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
47     _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
48
49     # --- unit parsing -----
50
51     def parse_size_mb(value_str: str, unit: str) -> int:
52         """Parse a size value + unit string into mebibytes."""

```

```

53     value = int(value_str)
54     unit = unit.lower().rstrip("b")
55     multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
56     if unit in multipliers:
57         raw = value * multipliers[unit]
58         if unit == "k":
59             raw = max(1, raw // 1024)
60         return raw
61     raise ValueError(f"Unknown size unit: {unit}")
62
63
64 def parse_duration_s(value_str: str, unit: str) -> int:
65     """Parse a duration value + unit string into seconds."""
66     value = int(value_str)
67     unit = unit.lower()
68     multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
69     if unit in multipliers:
70         return value * multipliers[unit]
71     raise ValueError(f"Unknown time unit: {unit}")
72
73
74 # --- formatting helpers -----
75
76
77 def _format_duration(seconds: int | None) -> str:
78     if seconds is None:
79         return "unlimited"
80     if seconds == 0:
81         return "0s"
82     if seconds < 60:
83         return f"{seconds}s"
84     if seconds < 3600:
85         m, s = divmod(seconds, 60)
86         return f"{m}m{s}s" if s else f"{m}m"
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     parts = f"{h}h"
90     if m:
91         parts += f"{m}m"
92     if s:
93         parts += f"{s}s"
94     return parts
95
96
97 def _format_size(mb: int | None) -> str:
98     if mb is None:
99         return "unlimited"
100     if mb == 0:
101         return "0MB"
102     if mb >= 1024:
103         gb = mb / 1024
104         if gb == int(gb):
105             return f"{int(gb)}GB"
106         return f"{gb:.1f}GB"
107     return f"{mb}MB"
108
109
110 def format_config(cfg: FilterConfig) -> str:
111     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
112     lines = [
113         "? **Current Filter Configuration**",
114         "",
115         f"? **Duration:** {_format_duration(cfg.min_duration_s)} -",
116         f"{_format_duration(cfg.max_duration_s)}",
117         f"? **Min resolution:** {'? ' + str(cfg.target_height) + 'p' if

```

```

cfg.target_height else 'disabled'}",
117         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
118         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
119         f"? **Delivery target:** `{cfg.delivery_target}`",
120         "",
121         "? **Content exclusions:**",
122         f"    GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
123         f"    Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
124         f"    Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
125     ]
126     return "\n".join(lines)
127
128
129     def format_commands() -> str:
130         """Return a help message listing all available commands."""
131         return "\n".join(
132             [
133                 "? **Available Commands**",
134                 "",
135                 "? **Filter config**",
136                 "`/show config` ? Show current filter settings",
137                 "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
138                 "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing",
139                 "`B`",
140                 "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
141                 "    or `/set max length <N><unit>` ? same as above",
142                 "    Units: `s` (default), `m`, `h` ? case-insensitive",
143                 "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length",
144                 "5s`)",
145                 "    Units: `s` (default), `m`, `h` ? case-insensitive",
146                 "`/set min resolution <N>p` ? Set min video height (e.g. `/set min",
147                 "resolution 720p`)",
148                 "    or `/set min res <N>p` ? same as above",
149                 "",
150                 "? **Content exclusions** (all off by default)",
151                 "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
152                 "`/set exclude round on|off` ? Exclude circular video messages",
153                 "`/set exclude nosound on|off` ? Exclude videos without audio\n",
154                 "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs",
155                 "too.",
156                 "",
157                 "? **Backfill**",
158                 "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history",
159                 "(default 90d)",
160                 "`/stop` ? Cancel running backfill",
161                 "",
162                 "\u2139ufe0f **Help**",
163                 "`/show commands` ? Show this help message",
164             ]
165         )
166
167     # --- command result -----
168
169     @dataclass
170     class CommandResult:
171         """The output of executing a command ? provider-independent.
172
173         The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
174         and ``config_changed`` to decide whether to hot-reload the filter engine.
175
176         """
177         reply_text: str = ""
178         config_changed: bool = False

```

```

176
177
178 # --- command service -----
179
180
181 class CommandService:
182     """Provider-independent command execution.
183
184     Parses a command string against an account's current config and DB,
185     returning a ``CommandResult``. No Telethon objects involved ? pure string
186     in, effects out. The Telethon adapter (``commands.py``) handles the
187     transport (checking message.out, sending the reply); the Bot API adapter
188     (future) calls ``handle()`` with the ``from.id`` from the update.
189
190     Usage::
191
192         svc = CommandService()
193         result = await svc.handle(user_id, text, conn, current_cfg)
194         if result.config_changed:
195             engine_holder["engine"] = FilterEngine(current_cfg)
196         await send_reply(result.reply_text)
197     """
198
199 # -- public API -----
200
201 async def handle(
202     self,
203     user_id: int,
204     text: str,
205     conn: aiosqlite.Connection,
206     current_cfg: FilterConfig,
207 ) -> CommandResult:
208     """Parse and execute a command text, returning the reply + change flag."""
209     text = text.rstrip()
210
211     # --- /set size <value><unit> -----
212     m = _SIZE_RE.match(text)
213     if m:
214         value_str, unit = m.group(1), m.group(2) or ""
215         # NOTE: ValueError is unreachable here ? the regex already validates
216         # the unit against [kmgT]?b?, so parse_size_mb can never raise.
217         # Kept for defensive parity with the original commands.py refactor.
218         try:
219             new_mb = parse_size_mb(value_str, unit)
220         except ValueError:
221             return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
222         if new_mb < 1:
223             return CommandResult("? Size must be at least 1 MB (``FilterConfig``
constraint).")
224         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
225         return CommandResult(
226             f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
227             config_changed=True,
228         )
229
230     # --- /set length <value><unit> -----
231     m = _LENGTH_RE.match(text)
232     if m:
233         value_str, unit = m.group(1), m.group(2) or ""
234         try:
235             new_secs = parse_duration_s(value_str, unit)
236         except ValueError:
237             return CommandResult("? Invalid duration format. Use e.g. `/set length

```

```

15m`.")
238         min_secs = current_cfg.min_duration_s or 0
239         if new_secs < min_secs:
240             return CommandResult(
241                 f"? Max duration ({_format_duration(new_secs)}) must be "
242                 f">= min duration ({_format_duration(min_secs)})."
243             )
244         cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
245         return CommandResult(
246             f"? Max duration set to
**{_format_duration(new_secs)}**.\n\n{format_config(cfg)}",
247             config_changed=True,
248         )
249
250     # --- /set min length <value><unit> -----
251     m = _MIN_LENGTH_RE.match(text)
252     if m:
253         value_str, unit = m.group(1), m.group(2) or ""
254         try:
255             new_min = parse_duration_s(value_str, unit)
256         except ValueError:
257             return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
258         max_secs = current_cfg.max_duration_s or 0
259         if max_secs > 0 and new_min > max_secs:
260             return CommandResult(
261                 f"? Min duration ({_format_duration(new_min)}) must be "
262                 f"<= max duration ({_format_duration(max_secs)})."
263             )
264         cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
265         return CommandResult(
266             f"? Min duration set to
**{_format_duration(new_min)}**.\n\n{format_config(cfg)}",
267             config_changed=True,
268         )
269
270     # --- /set min resolution <N>p -----
271     m = _MIN_RES_RE.match(text)
272     if m:
273         new_height = int(m.group(1))
274         if new_height < 1:
275             return CommandResult("? Resolution must be at least 1p.")
276         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
277         return CommandResult(
278             f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
279             config_changed=True,
280         )
281
282     if _MIN_RES_HELP_RE.match(text):
283         return CommandResult(
284             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
285         )
286
287     # --- /set exclude <gif|round|nosound> <on|off> -----
288     m = _EXCLUDE_RE.match(text)
289     if m:
290         what = m.group(1).lower()
291         state = m.group(2).lower()
292         enabled = state == "on"
293         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
294         field_map = {
295             "gif": "exclude_gif",

```

```

296         "round": "exclude_round",
297         "nosound": "exclude_nosound",
298     }
299     cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
300     return CommandResult(
301         f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
302         f"\n\n{format_config(cfg)}",
303         config_changed=True,
304     )
305
306     if _EXCLUDE_HELP_RE.match(text):
307         return CommandResult(
308             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
309             "Example: `/set exclude gif on`"
310         )
311
312     # --- /show config -----
313     if _SHOW_CONFIG_RE.match(text):
314         return CommandResult(format_config(current_cfg))
315
316     # --- source scanning commands (ADR 0009, Track E) -----
317
318     m = _WATCH_RE.match(text)
319     if m:
320         return await self._handle_watch(conn, user_id, m.group(1).strip(),
current_cfg)
321
322     m = _UNWATCH_RE.match(text)
323     if m:
324         return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
325
326     if _CHANNELS_RE.match(text):
327         return await self._handle_channels(conn, user_id)
328
329     if _WATCHLIST_RE.match(text):
330         return await self._handle_watchlist(conn, user_id)
331
332     # --- /show commands -----
333     if _SHOW_COMMANDS_RE.match(text):
334         return CommandResult(format_commands())
335
336     # Not a recognised command ? return empty result (noise).
337     return CommandResult()
338
339     # -- source scanning handlers (ADR 0009, Track E) -----
340
341     async def _handle_watch(
342         self, conn: aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
343     ) -> CommandResult:
344         """Parse `/watch <@username|channel_id>` and toggle `watched=1`."""
345         src = await self._resolve_source(conn, user_id, target)
346         if src is None:
347             return CommandResult(
348                 f"? Source `{target}` not found. Use `/channels` to list "
349                 f"available sources.\n\nSources are discovered as messages "
350                 f"arrive; a channel you haven't received traffic from may not "
351                 f"appear yet."
352             )
353         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]
354         label = src.title or src.provider_source_id
355         return CommandResult(f"? Now watching **{label}**"
(`{src.provider_source_id}`).")

```

```

356
357 async def _handle_unwatch(
358     self, conn: aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
359 ) -> CommandResult:
360     """Parse ``/unwatch <@username|channel_id>`` and toggle ``watched=0``."""
361     src = await self._resolve_source(conn, user_id, target)
362     if src is None:
363         return CommandResult(
364             f"? Source `{target}` not found. Use `/channels` to list sources."
365         )
366     await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
367     label = src.title or src.provider_source_id
368     return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
369
370 async def _handle_channels(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
371     """List all known sources with watch status."""
372     sources = await db.list_account_sources(conn, user_id)
373     if not sources:
374         return CommandResult(
375             "? **No sources yet.**\n\n"
376             "Sources are discovered when a new message arrives from a "
377             "channel. Run `/backfill` to discover historical channels, "
378             "or wait for new traffic."
379         )
380     lines = ["? **All sources**", ""]
381     for s in sources:
382         icon = "?" if s.watched else "?"
383         label = s.title or s.provider_source_id
384         lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385     return CommandResult("\n".join(lines))
386
387 async def _handle_watchlist(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
388     """List only watched sources."""
389     sources = await db.list_account_sources(conn, user_id, watched_only=True)
390     if not sources:
391         return CommandResult(
392             "? **No watched sources.**\n\n"
393             "Use `/channels` to see available sources, then `/watch <id>` "
394             "to start scanning."
395         )
396     lines = ["? **Watched sources**", ""]
397     for s in sources:
398         label = s.title or s.provider_source_id
399         lines.append(f"? `{s.provider_source_id}` ? {label}")
400     return CommandResult("\n".join(lines))
401
402 async def _resolve_source(
403     self, conn: aiosqlite.Connection, user_id: int, target: str
404 ) -> Source | None:
405     """Resolve a user-provided source identifier to a ``Source`` row.
406
407     Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
408     is stripped before lookup since Telegram channel ids are numeric.
409     """
410     target = target.lstrip("@")
411     # Try by provider-native id first (the common case).
412     src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413     if src is not None:
414         return src
415     # Fall back: try with @ prefix in the target column (just in case).
416     return await db.get_source_by_provider_id(conn, user_id, "telegram",

```

```

f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aiosqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,
432     ) -> FilterConfig:
433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s
439         if min_duration_s is not None:
440             updates["min_duration_s"] = min_duration_s
441         if target_height is not None:
442             updates["target_height"] = target_height
443         if exclude_gif is not None:
444             updates["exclude_gif"] = exclude_gif
445         if exclude_round is not None:
446             updates["exclude_round"] = exclude_round
447         if exclude_nosound is not None:
448             updates["exclude_nosound"] = exclude_nosound
449         cfg = current.model_copy(update=updates) if updates else current
450         await db.save_filter_config(conn, user_id, cfg)
451         log.info(
452             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
453             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
454             user_id,
455             cfg.max_size_mb,
456             cfg.min_duration_s,
457             cfg.max_duration_s,
458             cfg.target_height,
459             cfg.exclude_gif,
460             cfg.exclude_round,
461             cfg.exclude_nosound,
462         )
463         return cfg
464
465
466     __all__ = [
467         "ENGINE_KEY",
468         "CommandResult",
469         "CommandService",
470         "_format_duration",
471         "_format_size",
472         "format_config",
473         "format_commands",
474         "parse_duration_s",
475         "parse_size_mb",
476     ]
477

```

```

1      """Telethon ingestion: client factory, event handler, and metadata extraction.
2
3      Architecture (see docs/design.md §5):
4
5          NewMessage event
6              -> extract_video_meta(message)  # pure, unit-tested
7              -> dedup via db.get_video()
8              -> FilterEngine.evaluate(meta)
9              -> db.insert_video(...) + delivery.forward_to_target(...,
target=cfg.delivery_target)
10
11      The metadata extraction is deliberately a pure function taking a Telethon
12      ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
13      of logic that's worth unit-testing (everything else is Telethon/async glue).
14      """
15
16      from __future__ import annotations
17
18      import logging
19      import sqlite3
20      from pathlib import Path
21      from typing import Protocol
22
23      import aiosqlite
24      from telethon import TelegramClient
25      from telethon.events import NewMessage
26      from telethon.tl.types import (
27          Document,
28          DocumentAttributeAnimated,
29          DocumentAttributeVideo,
30          Message,
31          MessageMediaDocument,
32      )
33
34      from tg_video_filter import db
35      from tg_video_filter.commands import ENGINE_KEY
36      from tg_video_filter.delivery import build_link_text, deliver
37      from tg_video_filter.filters import FilterEngine, VideoMeta
38      from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User,
VideoRecord
39
40      log = logging.getLogger(__name__)
41
42
43      # --- pure metadata extraction -----
44
45
46      def extract_video_meta(message: Message) -> VideoMeta | None:
47          """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.
48
49          Returns ``None`` when the message has no video/document media. Returns a
50          ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
51          attributes are missing ? the filter engine decides how to treat gaps.
52
53          Telegram stores video metadata in two places:
54          - ``Document``: ``size`` (bytes), ``mime_type`` (str), ``attributes`` (list)
55          - ``DocumentAttributeVideo`` (inside ``attributes``): ``duration`` (float s),
56            ``w`` (width), ``h`` (height)
57
58          Note: we inspect ``message.media`` directly rather than the ``.video``
59          convenience property, because the latter only exists on Telethon's custom
60          Message wrapper (applied at runtime) and is ``None`` on the raw TL Message
61          used in tests. A Document is considered a video iff it contains a
62          ``DocumentAttributeVideo``.
63          """

```

```

64         media = getattr(message, "media", None)
65
66         # Real Telegram messages wrap documents in MessageMediaDocument.
67         # Unwrap it to get the actual Document.
68         if isinstance(media, MessageMediaDocument):
69             media = media.document
70
71         if not isinstance(media, Document):
72             return None
73
74         # Only treat this as a video if the document carries a video attribute.
75         has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in
media.attributes)
76         if not has_video_attr:
77             return None
78
79         meta = VideoMeta(
80             size_bytes=media.size,
81             mime_type=media.mime_type or None,
82         )
83
84         # Find the video attribute for duration/dimensions and round/nosound flags.
85         video_update: dict = {}
86         for attr in media.attributes:
87             if isinstance(attr, DocumentAttributeVideo):
88                 if attr.duration:
89                     video_update["duration_s"] = int(round(attr.duration))
90                 if attr.w:
91                     video_update["width"] = attr.w
92                 if attr.h:
93                     video_update["height"] = attr.h
94                 video_update["is_round"] = attr.round_message # may be None
95                 video_update["nosound"] = attr.nosound # may be None
96                 break
97
98         # Check for GIF (DocumentAttributeAnimated present).
99         is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)
100
101         meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
102         return meta
103
104
105 # --- client factory -----
106
107
108 def build_client(
109     session: TelegramSession, settings: SettingsLike
110 ) -> TelegramClient: # pragma: no cover
111     """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).
112
113     The session path is read from the ``telegram_sessions.session_path`` column
114     (data-driven, no longer derived from the PK at runtime) and resolved
115     against ``settings.sessions_dir``. The directory is created if missing.
116     """
117     sessions_dir = Path(settings.sessions_dir)
118     sessions_dir.mkdir(parents=True, exist_ok=True)
119     # session_path is stored as a basename (e.g. "user_111.session"); join with
120     # the runtime sessions_dir so the migration did not need to know the path.
121     session_path = sessions_dir / session.session_path
122
123     client = TelegramClient(
124         str(session_path),
125         api_id=settings.tg_api_id,
126         api_hash=settings.tg_api_hash,
127     )

```

```

128         return client
129
130
131     class SettingsLike(Protocol):
132         """Structural type for the subset of Settings the client factory needs.
133
134         Using a Protocol keeps the module importable in tests without dragging in
135         the real Settings class. At runtime ``config.Settings`` satisfies this shape.
136         """
137
138         tg_api_id: int
139         tg_api_hash: str
140         sessions_dir: str
141
142
143     # --- event handler wiring -----
144
145
146     def make_new_message_handler(
147         *,
148         user: User,
149         conn: aiosqlite.Connection,
150         engine_holder: dict[str, FilterEngine],
151         client: TelegramClient,
152         delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153         delivery_target: str = "me",
154     ) -> _NewMessageCallback:
155         """Return an async callback for ``events.NewMessage``.
156
157         The callback:
158         1. extracts VideoMeta + document_id (skips if not a video),
159         2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160            plus the existing (channel, message) instance dedup,
161         3. runs the filter engine (read from ``engine_holder`` so config
162            changes via ``/set`` take effect immediately),
163         4. inserts a VideoRecord row,
164         5. delivers a match to ``delivery_target`` (see ADR 0005).
165         """
166
167     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
168         message: Message = event.message # type: ignore[assignment]
169         if message is None:
170             return # Telethon edge case: event has no message (shouldn't happen)
171         # Only process genuine channel/group posts and private messages we
172         # receive ? outgoing messages (sent by our own account) are ignored.
173         if getattr(message, "out", False):
174             return
175
176         meta = extract_video_meta(message)
177         if meta is None:
178             return # not a video
179
180         channel_id = _peer_id(message.peer_id)
181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- watched gate (ADR 0009, Track E) -----
185         # Skip ingestion for unwatched sources. The channel must already be in
186         # sources (lazy-created by the /channels enumeration or the legacy
187         # upsert_channel below); if absent, we let the legacy path upsert it
188         # with watched=True for backwards-compatibility.
189         src = await db.get_source_by_provider_id(conn, user.id, "telegram",
190 str(channel_id))
191         if src is not None and not src.watched:
192             return # silently skip unwatched source

```

```

192
193     # --- three-tier dedup (cheapest first) ---
194
195     # Tier 0: same (user, channel, message) instance ? already processed.
196     existing = await db.get_video(conn, user.id, channel_id, message_id)
197     if existing is not None:
198         return
199
200     # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
201     if document_id is not None:
202         existing = await db.get_video_by_document_id(conn, user.id, document_id)
203         if existing is not None:
204             log.info(
205                 "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
206                 "original_channel=%s original_msg=%s",
207                 user.id,
208                 document_id,
209                 channel_id,
210                 message_id,
211                 existing.channel_id,
212                 existing.message_id,
213             )
214             return
215
216     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
217     if (
218         meta.size_bytes is not None
219         and meta.duration_s is not None
220         and meta.height is not None
221         and meta.width is not None
222     ):
223         existing = await db.get_video_by_content_fingerprint(
224             conn,
225             user.id,
226             meta.size_bytes,
227             meta.duration_s,
228             meta.height,
229             meta.width,
230         )
231         if existing is not None:
232             log.info(
233                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
234                 "channel=%s msg=%s original_channel=%s original_msg=%s",
235                 user.id,
236                 meta.size_bytes,
237                 meta.duration_s,
238                 meta.height,
239                 meta.width,
240                 channel_id,
241                 message_id,
242                 existing.channel_id,
243                 existing.message_id,
244             )
245             return
246
247     # Lazily record the channel (best-effort; failures are non-fatal).
248     try:
249         await db.upsert_channel(
250             conn,
251             Channel(user_id=user.id, channel_id=channel_id,
252 title=_channel_title(event)),
253         )
254     except Exception:
255         log.warning(
256             "failed to upsert channel %s for user %s", channel_id, user.id,

```

```

exc_info=True
256         )
257
258     result = engine_holder[ENGINE_KEY].evaluate(meta)
259     record = VideoRecord(
260         user_id=user.id,
261         channel_id=channel_id,
262         message_id=message_id,
263         document_id=document_id,
264         duration_s=meta.duration_s,
265         width=meta.width,
266         height=meta.height,
267         size_bytes=meta.size_bytes,
268         mime_type=meta.mime_type,
269         matched=result.matched,
270     )
271     try:
272         inserted = await db.insert_video(conn, record)
273     except sqlite3.IntegrityError:
274         # Lost a dedup race: a concurrent delivery of the same file
275         # (same document_id or same message instance) won the insert.
276         # The three-tier check above already classified this as new, so
277         # the collision means another task is processing (or has
278         # processed) it ? log and skip.
279         log.info(
280             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
281             user.id,
282             channel_id,
283             message_id,
284             document_id,
285         )
286     return
287     log.info(
288         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
289         user.id,
290         channel_id,
291         message_id,
292         result.matched,
293         result.reasons,
294     )
295
296     if result.matched:
297         # Build link text for LINK/BOTH modes (None for FORWARD).
298         link_text = None
299         if delivery_mode != DeliveryMode.FORWARD:
300             channel_title = _channel_title(event)
301             link_text = build_link_text(
302                 message_link=getattr(message, "link", None),
303                 duration_s=meta.duration_s,
304                 height=meta.height,
305                 size_bytes=meta.size_bytes,
306                 channel_title=channel_title,
307             )
308         await deliver(
309             client,
310             message,
311             conn,
312             inserted.id,
313             delivery_mode,
314             link_text=link_text,
315             target=delivery_target,
316         )
317
318     return on_new_message
319

```

```

320
321 # --- helpers -----
322
323
324 def _extract_document_id(message: Message) -> int | None:
325     """Extract Telegram's ``document.id`` from a video message.
326
327     Returns None if the message has no document media or the id is absent.
328     This is the most reliable content-based dedup key: Telegram reuses the
329     same ``document.id`` when a file is forwarded across channels.
330     """
331     media = getattr(message, "media", None)
332     if isinstance(media, MessageMediaDocument):
333         media = media.document
334     if isinstance(media, Document):
335         return getattr(media, "id", None)
336     return None
337
338
339 def _peer_id(peer: object) -> int:
340     """Best-effort numeric id for the peer of a message.
341
342     Telegram represents channel/group/user peers as different TL types; we
343     collapse them to a single int for dedup/storage. The exact subclass isn't
344     important ? only that the id is stable per chat.
345     """
346     # Each Peer* type has a field like channel_id / chat_id / user_id.
347     for field in ("channel_id", "chat_id", "user_id"):
348         val = getattr(peer, field, None)
349         if val is not None:
350             return int(val)
351     # Fallback: hash the repr so we never collide with real ids (negative).
352     return -abs(hash(repr(peer)) % (2**31))
353
354
355 def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
356     """Best-effort channel title from the event, if available."""
357     chat = getattr(event, "chat", None)
358     return getattr(chat, "title", None) if chat is not None else None
359
360
361 class _NewMessageCallback:
362     """Structural type annotation for the handler returned by make*_handler."""
363
364     async def __call__(self, event: NewMessage.Event) -> None: ...
365
366
367 __all__ = [
368     "build_client",
369     "extract_video_meta",
370     "make_new_message_handler",
371 ]
372

```

8. user (2026-07-06T09:07:26.538Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1389 of 1778 total (27190 tokens, cap 25000). Call Read with offset=1390 limit=1389 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1     """SQLite persistence layer (aiosqlite).
2
3     Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``

```

```

5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    from collections.abc import Awaitable, Callable, Iterable
15    from datetime import UTC, datetime
16
17    import aiosqlite
18
19    from tg_video_filter.models import (
20        Account,
21        Channel,
22        ContentItem,
23        DeliveryAttempt,
24        DeliveryStatus,
25        Destination,
26        FilterConfig,
27        Policy,
28        RoutingRule,
29        Source,
30        TelegramSession,
31        User,
32        VideoRecord,
33    )
34
35    # --- DDL -----
36
37    _SCHEMA = """
38    CREATE TABLE IF NOT EXISTS accounts (
39        id            INTEGER PRIMARY KEY,
40        telegram_id   INTEGER UNIQUE,
41        provider      TEXT NOT NULL DEFAULT 'telegram',
42        display_name  TEXT,
43        created_at    TEXT NOT NULL
44    );
45
46    CREATE TABLE IF NOT EXISTS telegram_sessions (
47        account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
48        phone         TEXT,
49        session_path  TEXT NOT NULL,
50        enabled       INTEGER NOT NULL DEFAULT 1,
51        created_at    TEXT NOT NULL,
52        last_login_at TEXT
53    );
54
55    CREATE TABLE IF NOT EXISTS channels (
56        id            INTEGER PRIMARY KEY AUTOINCREMENT,
57        user_id       INTEGER NOT NULL REFERENCES accounts(id),
58        channel_id    INTEGER NOT NULL,
59        title         TEXT,
60        seen_first_at TEXT NOT NULL,
61        UNIQUE (user_id, channel_id)
62    );
63
64    CREATE TABLE IF NOT EXISTS videos (
65        id            INTEGER PRIMARY KEY AUTOINCREMENT,
66        user_id       INTEGER NOT NULL REFERENCES accounts(id),
67        channel_id    INTEGER NOT NULL,
68        message_id    INTEGER NOT NULL,
69        document_id   INTEGER,

```

```

70     duration_s  INTEGER,
71     width       INTEGER,
72     height      INTEGER,
73     size_bytes  INTEGER,
74     mime_type   TEXT,
75     matched     INTEGER NOT NULL DEFAULT 0,
76     forwarded   INTEGER NOT NULL DEFAULT 0,
77     seen_at     TEXT NOT NULL,
78     UNIQUE (user_id, channel_id, message_id)
79 );
80
81 CREATE TABLE IF NOT EXISTS filter_config (
82     user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
83     config_json  TEXT NOT NULL
84 );
85
86 -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
87 -- ``content_items``, and ``policies`` are populated by migration 003 (and by
88 -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
89 -- are created empty here so the schema is complete ? Track C (#28) implements
90 -- the delivery logic that populates them.
91
92 CREATE TABLE IF NOT EXISTS sources (
93     id            INTEGER PRIMARY KEY AUTOINCREMENT,
94     account_id    INTEGER NOT NULL REFERENCES accounts(id),
95     provider      TEXT NOT NULL DEFAULT 'telegram',
96     provider_source_id TEXT NOT NULL,
97     title         TEXT,
98     kind          TEXT NOT NULL DEFAULT 'channel',
99     watched       INTEGER NOT NULL DEFAULT 0,
100    seen_first_at  TEXT NOT NULL,
101    last_seen_at   TEXT,
102    UNIQUE (account_id, provider, provider_source_id)
103 );
104
105 CREATE TABLE IF NOT EXISTS content_items (
106     id            INTEGER PRIMARY KEY AUTOINCREMENT,
107     account_id    INTEGER NOT NULL REFERENCES accounts(id),
108     source_id     INTEGER NOT NULL REFERENCES sources(id),
109     provider      TEXT NOT NULL DEFAULT 'telegram',
110     provider_message_id TEXT NOT NULL,
111     content_type  TEXT NOT NULL DEFAULT 'video',
112     text          TEXT,
113     link_url      TEXT,
114     media_ref     TEXT,
115     document_id   TEXT,
116     duration_s    INTEGER,
117     width         INTEGER,
118     height        INTEGER,
119     size_bytes    INTEGER,
120     mime_type     TEXT,
121     dedup_key     TEXT NOT NULL,
122     matched       INTEGER NOT NULL DEFAULT 0,
123     seen_at       TEXT NOT NULL,
124     UNIQUE (account_id, source_id, provider_message_id),
125     UNIQUE (account_id, dedup_key)
126 );
127
128 CREATE TABLE IF NOT EXISTS policies (
129     id            INTEGER PRIMARY KEY AUTOINCREMENT,
130     account_id    INTEGER NOT NULL REFERENCES accounts(id),
131     name          TEXT NOT NULL,
132     config_json   TEXT NOT NULL,
133     is_default    INTEGER NOT NULL DEFAULT 0,
134     created_at    TEXT NOT NULL,

```

```

135         updated_at TEXT NOT NULL,
136         UNIQUE (account_id, name)
137     );
138
139     -- ADR 0011: exactly one default policy per account (structural invariant).
140     CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
141     ON policies(account_id) WHERE is_default = 1;
142
143     CREATE TABLE IF NOT EXISTS destinations (
144         id INTEGER PRIMARY KEY AUTOINCREMENT,
145         account_id INTEGER NOT NULL REFERENCES accounts(id),
146         provider TEXT NOT NULL DEFAULT 'telegram',
147         target TEXT NOT NULL,
148         kind TEXT NOT NULL DEFAULT 'saved',
149         title TEXT,
150         is_output INTEGER NOT NULL DEFAULT 0,
151         created_at TEXT NOT NULL,
152         UNIQUE (account_id, provider, target)
153     );
154
155     CREATE TABLE IF NOT EXISTS routing_rules (
156         id INTEGER PRIMARY KEY AUTOINCREMENT,
157         account_id INTEGER NOT NULL REFERENCES accounts(id),
158         source_id INTEGER NOT NULL REFERENCES sources(id),
159         policy_id INTEGER REFERENCES policies(id),
160         destination_id INTEGER NOT NULL REFERENCES destinations(id),
161         enabled INTEGER NOT NULL DEFAULT 1,
162         priority INTEGER NOT NULL DEFAULT 0,
163         created_at TEXT NOT NULL,
164         UNIQUE (account_id, source_id, destination_id)
165     );
166
167     CREATE TABLE IF NOT EXISTS delivery_attempts (
168         id INTEGER PRIMARY KEY AUTOINCREMENT,
169         content_item_id INTEGER NOT NULL REFERENCES content_items(id),
170         destination_id INTEGER NOT NULL REFERENCES destinations(id),
171         route_id INTEGER REFERENCES routing_rules(id),
172         status TEXT NOT NULL DEFAULT 'pending',
173         attempt_count INTEGER NOT NULL DEFAULT 0,
174         last_error TEXT,
175         created_at TEXT NOT NULL,
176         delivered_at TEXT,
177         next_retry_at TEXT,
178         UNIQUE (content_item_id, destination_id)
179     );
180     """
181
182     # Note: the ``idx_videos_user_document`` partial unique index is created by
183     # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
184     # which is present on fresh tables but only added to legacy v1 DBs by the
185     # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
186     # DBs that don't yet have the column.
187
188
189     async def init_db(db_path: str) -> None:
190         """Open the database at ``db_path`` and create/migrate the schema.
191
192         Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
193         then runs any pending migrations via the ``user_version`` ladder.
194         """
195         async with aiosqlite.connect(db_path) as conn:
196             await _apply_schema(conn)
197
198
199     async def _apply_schema(conn: aiosqlite.Connection) -> None:

```

```

200 # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
201 # and a busy timeout so concurrent access waits instead of failing immediately.
202 await conn.execute("PRAGMA journal_mode = WAL")
203 await conn.execute("PRAGMA foreign_keys = ON")
204 await conn.execute("PRAGMA busy_timeout = 5000")
205 await conn.executescript(_SCHEMA)
206 # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
207 # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
208 # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
209 # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
210 # any *empty* recreations of those legacy tables from a subsequent
211 # ``init_db`` are dropped here so they don't shadow the renamed copies.
212 if await _table_exists(conn, "channels_deprecated"):
213     await conn.execute("DROP TABLE IF EXISTS channels")
214     await conn.execute("DROP TABLE IF EXISTS videos")
215     await conn.execute("DROP TABLE IF EXISTS filter_config")
216 await _apply_migrations(conn)
217 await conn.commit()
218
219
220 # --- migrations -----
221 #
222 # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
223 # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
224 # the source of truth for *fresh* databases; the migrations below reconcile
225 # *existing* databases that were created by an older schema version.
226 #
227 # To add a new migration:
228 # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
229 # 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
230 # 3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
231 #    ``init_db``, and asserts the new structure.
232
233 _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
234
235
236 def _migration(
237     version: int,
238 ) -> Callable[
239     [Callable[[aiosqlite.Connection], Awaitable[None]]],
240     Callable[[aiosqlite.Connection], Awaitable[None]],
241 ]:
242     """Register a migration that brings the DB up to ``user_version == version``."""
243
244     def decorator(
245         fn: Callable[[aiosqlite.Connection], Awaitable[None]],
246     ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
247         _MIGRATIONS.append((version, fn))
248         return fn
249
250     return decorator
251
252
253 # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
254 #
255 # Pure function: computes the ``content_items.dedup_key`` value from metadata
256 # already extracted at ingestion time (no download ? honors ADR 0003). The
257 # decision tree is ordered strongest-key first; each branch is guarded so
258 # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0``).
259 # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
260
261
262 def compute_dedup_key(
263     *,
264     provider: str,

```

```

265     document_id: str | None = None,
266     source_id: int | str | None = None,
267     provider_message_id: str | None = None,
268     size_bytes: int | None = None,
269     duration_s: int | None = None,
270     height: int | None = None,
271     width: int | None = None,
272 ) -> str:
273     """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
274
275     Implements the ADR 0011 Option 1A decision tree:
276
277     - **Telegram** (``provider == "telegram"``):
278       1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
279       2. Else if **all four** fingerprint fields (size, duration, height, width)
280          are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
281       3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
282          ``source_id`` is required because Telegram ``message_id`` is only
283          unique within a channel).
284     - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
285       ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
286       providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
287
288     Raises ``ValueError`` if the fallback branch is reached without the fields
289     it requires (``source_id`` + ``provider_message_id``) ? a degenerate
290     ``dedup_key`` is worse than a loud failure at ingestion time.
291     """
292     p = provider.lower()
293     if p == "telegram":
294         if document_id is not None and document_id != "":
295             return f"tg:doc:{document_id}"
296         # All four fingerprint fields are required to avoid a degenerate
297         # ``tg:fp:0:0:0:0`` key when some metadata is missing.
298         if None not in (size_bytes, duration_s, height, width):
299             return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
300         # Tier-0 fallback.
301         if source_id is None or provider_message_id is None:
302             raise ValueError(
303                 "Telegram dedup_key fallback requires source_id and provider_message_id"
304             )
305         return f"tg:msg:{source_id}:{provider_message_id}"
306     # Future-proof default for providers without a dedicated branch.
307     if source_id is None or provider_message_id is None:
308         raise ValueError(
309             f"dedup_key for provider={provider!r} requires source_id and"
310             f"provider_message_id"
311         )
312     return f"{p}:msg:{source_id}:{provider_message_id}"
313
314
315 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
316     """Run every registered migration whose version > current ``user_version``."""
317     async with conn.execute("PRAGMA user_version") as cur:
318         row = await cur.fetchone()
319         current = int(row[0]) if row else 0
320
321     for version, fn in _MIGRATIONS:
322         if version <= current:
323             continue
324         await fn(conn)
325         await conn.execute(f"PRAGMA user_version = {version}")
326
327

```

```

328 async def _column_exists(conn: aiomysqlite.Connection, table: str, column: str) -> bool:
329     """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
330     async with conn.execute(f"PRAGMA table_info({table})") as cur:
331         rows = await cur.fetchall()
332         return any(row[1] == column for row in rows)
333
334
335 @_migration(1)
336 async def _migration_001_add_document_id(conn: aiomysqlite.Connection) -> None:
337     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
338
339     Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
340     created before this migration existed.
341     """
342     if not await _column_exists(conn, "videos", "document_id"):
343         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
344     # Create the partial unique index for both fresh and migrated DBs. Uses
345     # IF NOT EXISTS so it is safe on DBs where it was already created.
346     await conn.execute(
347         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
348         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
349     )
350
351
352 @_migration(2)
353 async def _migration_002_identity_split(conn: aiomysqlite.Connection) -> None:
354     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
355
356     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
357     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
358     so their integer ``user_id`` values still resolve to the same identity (now
359     via ``accounts.id``). The legacy ``users`` table is renamed to
360     ``users_deprecated`` and kept for one release as a rollback safety net; a
361     follow-up migration drops it.
362
363     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
364     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
365     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
366     those FKs to point at ``users_deprecated``, diverging from fresh installs
367     and breaking inserts for new accounts ? see PR #33 review.)
368
369     Idempotent: a no-op once the legacy ``users`` table has been renamed.
370     """
371     # If the legacy users table is gone, this migration has already run
372     # (or the DB is fresh and only has accounts). Nothing to do.
373     if not await _table_exists(conn, "users"):
374         return
375
376     # Build accounts from the legacy users table. accounts.id := users.id.
377     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
378     # and recreated here so the INSERT can populate it cleanly.)
379     await conn.execute("DROP TABLE IF EXISTS accounts")
380     await conn.execute(
381         """
382         CREATE TABLE accounts (
383             id            INTEGER PRIMARY KEY,
384             telegram_id   INTEGER UNIQUE,
385             provider      TEXT NOT NULL DEFAULT 'telegram',
386             display_name  TEXT,
387             created_at    TEXT NOT NULL
388         )
389         """
390     )
391     await conn.execute(
392         """

```

```

393         INSERT INTO accounts (id, telegram_id, provider, created_at)
394         SELECT id, id, 'telegram', created_at FROM users
395         """"
396     )
397
398     # Build telegram_sessions, one row per legacy userbot user.
399     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
400     await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
401     await conn.execute(
402         """"
403         CREATE TABLE telegram_sessions (
404             account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
405             phone          TEXT,
406             session_path   TEXT NOT NULL,
407             enabled        INTEGER NOT NULL DEFAULT 1,
408             created_at     TEXT NOT NULL,
409             last_login_at  TEXT
410         )
411         """"
412     )
413     # session_path is built from the *telegram_id* (= old users.id) to match
414     # the convention telethon_client.build_client used pre-migration. The
415     # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
416     # so this migration does not need to know the runtime path.
417     await conn.execute(
418         """"
419         INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
420 created_at)
421         SELECT id, phone, 'user_' || id || '.session', enabled, created_at
422         FROM users
423         """"
424     )
425
426     # Rebuild channels / videos / filter_config so their FK constraints point
427     # at accounts(id) instead of the legacy users(id). SQLite's
428     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
429     # the rename ? so a naive rename of users ? users_deprecated would leave
430     # these tables referencing users_deprecated(id), diverging from fresh DBs
431     # (which reference accounts(id)) and breaking inserts for new accounts.
432     # The rebuild: rename the data-holding legacy table out of the way, create
433     # the new one with the correct FK, copy the data back, drop the renamed copy.
434     for table, new_ddl, copy_cols in (
435         (
436             "channels",
437             """"
438             CREATE TABLE channels (
439                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
440                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
441                 channel_id    INTEGER NOT NULL,
442                 title         TEXT,
443                 seen_first_at TEXT NOT NULL,
444                 UNIQUE (user_id, channel_id)
445             )
446             """,
447             "id, user_id, channel_id, title, seen_first_at",
448         ),
449         (
450             "videos",
451             """"
452             CREATE TABLE videos (
453                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
454                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
455                 channel_id    INTEGER NOT NULL,
456                 message_id    INTEGER NOT NULL,
457                 document_id   INTEGER,

```

```

457         duration_s    INTEGER,
458         width         INTEGER,
459         height        INTEGER,
460         size_bytes    INTEGER,
461         mime_type     TEXT,
462         matched       INTEGER NOT NULL DEFAULT 0,
463         forwarded     INTEGER NOT NULL DEFAULT 0,
464         seen_at       TEXT NOT NULL,
465         UNIQUE (user_id, channel_id, message_id)
466     )
467     """
468     "id, user_id, channel_id, message_id, document_id, duration_s, width, "
469     "height, size_bytes, mime_type, matched, forwarded, seen_at",
470 ),
471 (
472     "filter_config",
473     """
474     CREATE TABLE filter_config (
475         user_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
476         config_json TEXT NOT NULL
477     )
478     """
479     "user_id, config_json",
480 ),
481 ):
482     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
483     await conn.execute(new_ddl)
484     await conn.execute(
485         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
486     )
487     await conn.execute(f"DROP TABLE {table}__legacy_data")
488
489     # The partial unique index for document_id dedup (created by migration 001
490     # against the legacy videos table) was dropped along with the legacy copy
491     # above. Recreate it on the new videos table.
492     await conn.execute(
493         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
494         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
495     )
496
497     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
498     # net for one release per ADR 0010). The data-holding channels/videos/
499     # filter_config tables were already rebuilt above with FKs to accounts(id),
500     # so renaming users no longer retargets any live FK.
501     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
502
503
504 @migration(3)
505 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
506     """Land the provider-neutral domain tables (ADR 0011, Track B).
507
508     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
509     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
510     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
511     them), then backfills the three populated ones from the legacy
512     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
513     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
514     pattern). Idempotent: a no-op once the legacy tables have been renamed.
515     """
516     # Idempotency gate: if the legacy videos table is gone, this migration has
517     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
518     # dropped by _apply_schema post-003). Nothing to backfill.
519     if not await _table_exists(conn, "videos"):
520         return

```

```

521
522 # 1a. Backfill sources from channels first (known title + watched=1). Order
523 # matters: channels-first so title/watched data wins; the videos-based
524 # backfill below fills gaps for orphan channels (review finding #1).
525 await conn.execute(
526     """
527     INSERT OR IGNORE INTO sources
528         (account_id, provider, provider_source_id, title, kind, watched,
529          seen_first_at, last_seen_at)
530     SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
531            1, seen_first_at, seen_first_at
532     FROM channels
533     """
534 )
535
536 # 1b. Backfill sources from videos for channels that have no channels row
537 # (e.g. backfill-only channels that were never seen via live traffic).
538 # INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
539 # ? the channels-based row above survives with its title + watched=1.
540 await conn.execute(
541     """
542     INSERT OR IGNORE INTO sources
543         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
544     SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
545            'channel', 0, MIN(v.seen_at)
546     FROM videos v
547     GROUP BY v.user_id, v.channel_id
548     """
549 )
550
551 # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
552 # Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
553 # (no silent drops, even on fingerprint collisions).
554 # Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
555 group).
556 # Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
557 group).
558 # The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
559 await conn.execute(
560     """
561     INSERT INTO content_items
562         (account_id, source_id, provider, provider_message_id, content_type,
563          document_id, duration_s, width, height, size_bytes, mime_type,
564          dedup_key, matched, seen_at)
565     SELECT
566         v.user_id,
567         s.id,
568         'telegram',
569         CAST(v.message_id AS TEXT),
570         'video',
571         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
572         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
573         'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
574         v.matched,
575         v.seen_at
576     FROM videos v
577     JOIN sources s
578         ON s.account_id = v.user_id
579         AND s.provider = 'telegram'
580         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
581     """
582 )
583
584 # Phase B: promote one row per (account, document_id) to tg:doc:.
585 await conn.execute(

```

```

584         """
585         UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
586         WHERE document_id IS NOT NULL
587             AND dedup_key LIKE 'tg:msg:%'
588             AND id IN (
589             SELECT MIN(id) FROM content_items
590             WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
591             GROUP BY account_id, document_id
592         )
593         """
594     )
595
596     # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
597     await conn.execute(
598         """
599         UPDATE content_items SET dedup_key =
600             'tg:fp:' || size_bytes || ':' || duration_s || ':'
601             || height || ':' || width
602         WHERE document_id IS NULL
603             AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
604             AND height IS NOT NULL AND width IS NOT NULL
605             AND dedup_key LIKE 'tg:msg:%'
606             AND id IN (
607             SELECT MIN(id) FROM content_items
608             WHERE document_id IS NULL
609                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
610                 AND height IS NOT NULL AND width IS NOT NULL
611                 AND dedup_key LIKE 'tg:msg:%'
612             GROUP BY account_id, size_bytes, duration_s, height, width
613         )
614         """
615     )
616
617     # 3. Backfill policies from filter_config: one 'default' policy per account
618     #     (Option 2A). is_default=1 so the legacy load_filter_config shim reads
619     #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
620     await conn.execute(
621         """
622         INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
623                                         created_at, updated_at)
624         SELECT user_id, 'default', config_json, 1,
625             strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
626             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
627         FROM filter_config
628         """
629     )
630
631     # 4. Create the policies default-partial index (sig after backfill so the
632     #     UNIQUE constraint only applies once rows are populated). On fresh DBs
633     #     _SCHEMA creates it; on migrated DBs this is the first time.
634     await conn.execute(
635         "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
636         "ON policies(account_id) WHERE is_default = 1"
637     )
638
639     # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
640     #     produce a content_items row. A mismatch signals fingerprint collisions
641     #     where two distinct videos shared (size,dur,h,w) ? the losers stay at
642     #     tg:msg: keys and are still present. Log a warning so the operator can
643     #     investigate; the data is preserved in *_deprecated tables.
644     async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
645         vc = (await cur.fetchone())[0]
646     async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
647         cic = (await cur.fetchone())[0]
648     if vc != cic:

```

```

649         import logging
650
651         _logger = logging.getLogger(__name__)
652         _logger.warning(
653             "migration_003: %d videos rows produced %d content_items rows "
654             "%d gap). Colliding items remain in videos_deprecated.",
655             vc,
656             cic,
657             vc - cic,
658         )
659
660         # 6. Rename (don't drop) the legacy tables ? kept for one release as a
661         #     rollback safety net (ADR 0010 pattern).
662         await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
663         await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
664         await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
665
666
667     @_migration(4)
668     async def _migration_004_destinations_and_routing(conn: aiomysqlite.Connection) -> None:
669         """Backfill default destinations and delivery state (ADR 0011, Track C).
670
671         Creates a default destination row for each account that has a
672         ``filter_config_deprecated.delivery_target`` value, then creates a default
673         routing rule (source ? default policy ? destination) for each source.
674         Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675         table (one ``delivered`` attempt per previously-forwarded video to the
676         account's default destination).
677
678         Idempotent: skips accounts whose default destination already exists.
679         """
680         # Gate: if destinations already have data, this migration has run.
681         async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682             if (await cur.fetchone())[0] > 0:
683                 return
684
685         if not await _table_exists(conn, "filter_config_deprecated"):
686             return
687
688         # 1. Backfill default destinations from filter_config_deprecated.
689         #     Parse the JSON delivery_target, defaulting to "me".
690         await conn.execute(
691             """
692             INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
693             is_output, created_at)
694             SELECT user_id, 'telegram',
695                 COALESCE(
696                     CASE WHEN json_valid(config_json)
697                         THEN json_extract(config_json, '$.delivery_target') END,
698                     'me'
699                 ),
700                 CASE
701                     WHEN COALESCE(
702                         CASE WHEN json_valid(config_json)
703                             THEN json_extract(config_json, '$.delivery_target') END,
704                         'me'
705                     ) = 'me'
706                     THEN 'saved'
707                     ELSE 'channel'
708                 END,
709                 0,
710                 strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
711             FROM filter_config_deprecated
712             """
713         )

```

```

713
714     # 2. Create a default routing rule per source (source ? default policy ? default
destination).
715     await conn.execute(
716         """
717         INSERT OR IGNORE INTO routing_rules
718         (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
719         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
720             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
721         FROM sources s
722         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
723         JOIN destinations d ON d.account_id = s.account_id
724         WHERE d.target = (
725             SELECT COALESCE(
726                 CASE WHEN json_valid(fcd.config_json)
727                     THEN json_extract(fcd.config_json, '$.delivery_target') END,
728                 'me'
729             )
730             FROM filter_config_deprecated fcd
731             WHERE fcd.user_id = s.account_id
732             LIMIT 1
733         )
734         """
735     )
736
737     # 3. Backfill delivery_attempts from previously-forwarded videos.
738     await conn.execute(
739         """
740         INSERT OR IGNORE INTO delivery_attempts
741         (content_item_id, destination_id, route_id, status, attempt_count,
created_at, delivered_at)
742         SELECT ci.id, d.id, rr.id, 'delivered', 1,
743             vd.seen_at, vd.seen_at
744         FROM videos_deprecated vd
745         JOIN content_items ci
746             ON ci.account_id = vd.user_id
747             AND ci.provider = 'telegram'
748             AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
749         JOIN sources sc
750             ON sc.id = ci.source_id
751             AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
752         JOIN destinations d ON d.account_id = vd.user_id
753         JOIN routing_rules rr
754             ON rr.account_id = vd.user_id
755             AND rr.source_id = sc.id
756             AND rr.destination_id = d.id
757         WHERE vd.forwarded = 1
758         """
759     )
760
761
762
763     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
764         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
765         async with conn.execute(
766             "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
767         ) as cur:
768             return await cur.fetchone() is not None
769
770
771     # --- helpers -----
772
773
774     def _now() -> datetime:
775         """Timezone-aware UTC 'now', used for policy timestamps."""

```

```

776         return datetime.now(UTC)
777
778
779 def _safe_int_cast(value: object, field_name: str) -> int:
780     """Cast a provider_source_id / provider_message_id / document_id to int.
781
782     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
783     providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
784     shapes (which carry Telegram-specific int ids); this function raises a clear
785     ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
786     (ADR 0011 provider-neutrality guard ? review finding #5).
787     """
788     try:
789         return int(value) # type: ignore[arg-type]
790     except (ValueError, TypeError) as exc:
791         raise ValueError(
792             f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
793             "
794             f"types are Telegram-specific; use ContentItem/Source directly for "
795             f"non-Telegram providers (ADR 0011)."
796         ) from exc
797
798 def _row_to_account(row: aiosqlite.Row) -> Account:
799     return Account(
800         id=row["id"],
801         telegram_id=row["telegram_id"],
802         provider=row["provider"],
803         display_name=row["display_name"],
804         created_at=row["created_at"], # stored as ISO string; pydantic parses
805     )
806
807
808 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
809     return TelegramSession(
810         account_id=row["account_id"],
811         phone=row["phone"],
812         session_path=row["session_path"],
813         enabled=bool(row["enabled"]),
814         created_at=row["created_at"],
815         last_login_at=row["last_login_at"],
816     )
817
818
819 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
820     """Project an (account, optional session) pair onto the legacy ``User`` view.
821
822     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
823     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
824     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
825     "list enabled users" semantics exclude it ? matching the structural rule
826     that bot-only users are not runnable Telethon clients.
827     """
828     if session is None:
829         return User(
830             id=account.telegram_id if account.telegram_id is not None else account.id,
831             phone=None,
832             enabled=False,
833             created_at=account.created_at,
834         )
835     return User(
836         id=account.telegram_id if account.telegram_id is not None else account.id,
837         phone=session.phone,
838         enabled=session.enabled,
839         created_at=session.created_at,

```

```

840     )
841
842
843 def _row_to_source(row: aiosqlite.Row) -> Source:
844     return Source(
845         id=row["id"],
846         account_id=row["account_id"],
847         provider=row["provider"],
848         provider_source_id=row["provider_source_id"],
849         title=row["title"],
850         kind=row["kind"],
851         watched=bool(row["watched"]),
852         seen_first_at=row["seen_first_at"],
853         last_seen_at=row["last_seen_at"],
854     )
855
856
857 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
858     return ContentItem(
859         id=row["id"],
860         account_id=row["account_id"],
861         source_id=row["source_id"],
862         provider=row["provider"],
863         provider_message_id=row["provider_message_id"],
864         content_type=row["content_type"],
865         text=row["text"],
866         link_url=row["link_url"],
867         media_ref=row["media_ref"],
868         document_id=row["document_id"],
869         duration_s=row["duration_s"],
870         width=row["width"],
871         height=row["height"],
872         size_bytes=row["size_bytes"],
873         mime_type=row["mime_type"],
874         dedup_key=row["dedup_key"],
875         matched=bool(row["matched"]),
876         seen_at=row["seen_at"],
877     )
878
879
880 def _row_to_policy(row: aiosqlite.Row) -> Policy:
881     return Policy(
882         id=row["id"],
883         account_id=row["account_id"],
884         name=row["name"],
885         config_json=row["config_json"],
886         is_default=bool(row["is_default"]),
887         created_at=row["created_at"],
888         updated_at=row["updated_at"],
889     )
890
891
892 def _row_to_destination(row: aiosqlite.Row) -> Destination:
893     return Destination(
894         id=row["id"],
895         account_id=row["account_id"],
896         provider=row["provider"],
897         target=row["target"],
898         kind=row["kind"],
899         title=row["title"],
900         is_output=bool(row["is_output"]),
901         created_at=row["created_at"],
902     )
903
904

```

```

905 def _row_to_routing_rule(row: aiosqlite.Row) -> RoutingRule:
906     return RoutingRule(
907         id=row["id"],
908         account_id=row["account_id"],
909         source_id=row["source_id"],
910         policy_id=row["policy_id"],
911         destination_id=row["destination_id"],
912         enabled=bool(row["enabled"]),
913         priority=row["priority"],
914         created_at=row["created_at"],
915     )
916
917
918 def _row_to_delivery_attempt(row: aiosqlite.Row) -> DeliveryAttempt:
919     return DeliveryAttempt(
920         id=row["id"],
921         content_item_id=row["content_item_id"],
922         destination_id=row["destination_id"],
923         route_id=row["route_id"],
924         status=DeliveryStatus(row["status"]),
925         attempt_count=row["attempt_count"],
926         last_error=row["last_error"],
927         created_at=row["created_at"],
928         delivered_at=row["delivered_at"],
929         next_retry_at=row["next_retry_at"],
930     )
931
932
933 # --- account + session repos (ADR 0010) -----
934
935
936 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
937     """Insert or update an account row; returns the persisted Account."""
938     await conn.execute(
939         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
940         VALUES (?, ?, ?, ?, ?)
941         ON CONFLICT(id) DO UPDATE SET
942             telegram_id=excluded.telegram_id,
943             provider=excluded.provider,
944             display_name=excluded.display_name""",
945         (
946             account.id,
947             account.telegram_id,
948             account.provider,
949             account.display_name,
950             account.created_at.isoformat(),
951         ),
952     )
953     await conn.commit()
954     return await get_account(conn, account.id) # type: ignore[return-value]
955
956
957 async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
958     conn.row_factory = aiosqlite.Row
959     async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as
960     cur:
961         row = await cur.fetchone()
962         return _row_to_account(row) if row else None
963
964
965 async def get_account_by_telegram_id(
966     conn: aiosqlite.Connection, telegram_id: int
967 ) -> Account | None:
968     """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
969     conn.row_factory = aiosqlite.Row

```

```

969         async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
(telegram_id,)) as cur:
970             row = await cur.fetchone()
971             return _row_to_account(row) if row else None
972
973
974     async def upsert_telegram_session(
975         conn: aiomysqlite.Connection, session: TelegramSession
976     ) -> TelegramSession:
977         """Insert or update a telegram_sessions row; returns the persisted session."""
978         await conn.execute(
979             """INSERT INTO telegram_sessions
980                 (account_id, phone, session_path, enabled, created_at, last_login_at)
981                 VALUES (?, ?, ?, ?, ?, ?)
982                 ON CONFLICT(account_id) DO UPDATE SET
983                     phone=excluded.phone,
984                     session_path=excluded.session_path,
985                     enabled=excluded.enabled,
986                     last_login_at=excluded.last_login_at""",
987             (
988                 session.account_id,
989                 session.phone,
990                 session.session_path,
991                 int(session.enabled),
992                 session.created_at.isoformat(),
993                 session.last_login_at.isoformat() if session.last_login_at else None,
994             ),
995         )
996         await conn.commit()
997         conn.row_factory = aiomysqlite.Row
998         async with conn.execute(
999             "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
1000         ) as cur:
1001             row = await cur.fetchone()
1002             # The row must exist ? we just upserted it by PK (account_id). Assert the
1003             # invariant explicitly so mypy can narrow Row | None ? Row without a
1004             # type-ignore comment masking a real future regression.
1005             assert row is not None
1006             return _row_to_telegram_session(row)
1007
1008
1009     async def get_telegram_session(
1010         conn: aiomysqlite.Connection, account_id: int
1011     ) -> TelegramSession | None:
1012         conn.row_factory = aiomysqlite.Row
1013         async with conn.execute(
1014             "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
1015         ) as cur:
1016             row = await cur.fetchone()
1017             return _row_to_telegram_session(row) if row else None
1018
1019
1020     async def list_runnable_sessions(
1021         conn: aiomysqlite.Connection,
1022     ) -> list[tuple[Account, TelegramSession]]:
1023         """Return (account, session) pairs for sessions the manager should start.
1024
1025         This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
1026         not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
1027         therefore inherently invisible to the Telethon supervisor ? the "don't
1028         start a client for bot-only users" invariant is enforced by query shape.
1029         """
1030         conn.row_factory = aiomysqlite.Row
1031         async with conn.execute(
1032             """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,

```

```

1033         s.created_at AS s_created_at, s.last_login_at
1034     FROM accounts a
1035     JOIN telegram_sessions s ON s.account_id = a.id
1036     WHERE s.enabled = 1""
1037 ) as cur:
1038     rows = await cur.fetchall()
1039     return [
1040         (
1041             _row_to_account(r),
1042             TelegramSession(
1043                 account_id=r["id"],
1044                 phone=r["s_phone"],
1045                 session_path=r["session_path"],
1046                 enabled=bool(r["s_enabled"]),
1047                 created_at=r["s_created_at"],
1048                 last_login_at=r["last_login_at"],
1049             ),
1050         )
1051         for r in rows
1052     ]
1053
1054
1055 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
1056
1057
1058 async def upsert_user(conn: aiomysqlite.Connection, user: User) -> User:
1059     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
1060
1061     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
1062     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
1063     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
1064     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
1065     """
1066     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
1067     await upsert_account(conn, account)
1068     await upsert_telegram_session(
1069         conn,
1070         TelegramSession(
1071             account_id=user.id,
1072             phone=user.phone,
1073             session_path=f"user_{user.id}.session",
1074             enabled=user.enabled,
1075             created_at=user.created_at,
1076         ),
1077     )
1078     return await get_user(conn, user.id) # type: ignore[return-value]
1079
1080
1081 async def get_user(conn: aiomysqlite.Connection, user_id: int) -> User | None:
1082     """Deprecated shim: project the (account, session) pair back to a ``User``."""
1083     account = await get_account(conn, user_id)
1084     if account is None:
1085         return None
1086     session = await get_telegram_session(conn, user_id)
1087     return _account_session_to_user(account, session)
1088
1089
1090 async def list_enabled_users(conn: aiomysqlite.Connection) -> list[User]:
1091     """Deprecated shim: list users that have an enabled MTPROTO session.
1092
1093     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
1094     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
1095     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
1096     """
1097     pairs = await list_runnable_sessions(conn)

```

```

1098         return [_account_session_to_user(a, s) for a, s in pairs]
1099
1100
1101 # --- Source repos (ADR 0011) -----
1102
1103
1104 async def upsert_source(conn: aisqlite.Connection, source: Source) -> Source:
1105     """Insert a source row, or update title/watched on conflict.
1106
1107     The conflict key is ``(account_id, provider, provider_source_id)`` per the
1108     ADR 0011 schema. Returns the persisted row.
1109     """
1110     await conn.execute(
1111         """INSERT INTO sources
1112             (account_id, provider, provider_source_id, title, kind, watched,
1113              seen_first_at, last_seen_at)
1114             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116                 title=excluded.title,
1117                 watched=excluded.watched,
1118                 last_seen_at=excluded.last_seen_at""",
1119         (
1120             source.account_id,
1121             source.provider,
1122             source.provider_source_id,
1123             source.title,
1124             source.kind,
1125             int(source.watched),
1126             source.seen_first_at.isoformat(),
1127             (source.last_seen_at.isoformat() if source.last_seen_at else None),
1128         ),
1129     )
1130     await conn.commit()
1131     conn.row_factory = aisqlite.Row
1132     async with conn.execute(
1133         """SELECT * FROM sources
1134             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1135         (source.account_id, source.provider, source.provider_source_id),
1136     ) as cur:
1137         row = await cur.fetchone()
1138     assert row is not None # just upserted by the conflict key
1139     return _row_to_source(row)
1140
1141
1142 async def get_source_by_provider_id(
1143     conn: aisqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144 ) -> Source | None:
1145     """Resolve a source by its provider-native id."""
1146     conn.row_factory = aisqlite.Row
1147     async with conn.execute(
1148         """SELECT * FROM sources
1149             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1150         (account_id, provider, provider_source_id),
1151     ) as cur:
1152         row = await cur.fetchone()
1153     return _row_to_source(row) if row else None
1154
1155
1156 # --- ContentItem repos (ADR 0011) -----
1157
1158
1159 async def insert_content_item(conn: aisqlite.Connection, item: ContentItem) ->
ContentItem:
1160     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1161

```

```

1162 The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1163 unique constraints guard against duplicates:
1164 - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1165 - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1166 """
1167 cur = await conn.execute(
1168     """INSERT INTO content_items
1169         (account_id, source_id, provider, provider_message_id, content_type,
1170          text, link_url, media_ref, document_id, duration_s, width, height,
1171          size_bytes, mime_type, dedup_key, matched, seen_at)
1172         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1173     (
1174         item.account_id,
1175         item.source_id,
1176         item.provider,
1177         item.provider_message_id,
1178         item.content_type,
1179         item.text,
1180         item.link_url,
1181         item.media_ref,
1182         item.document_id,
1183         item.duration_s,
1184         item.width,
1185         item.height,
1186         item.size_bytes,
1187         item.mime_type,
1188         item.dedup_key,
1189         int(item.matched),
1190         item.seen_at.isoformat(),
1191     ),
1192 )
1193 await conn.commit()
1194 return item.model_copy(update={"id": cur.lastrowid})
1195
1196
1197 async def get_content_item_by_message(
1198     conn: aiosqlite.Connection,
1199     account_id: int,
1200     source_id: int,
1201     provider_message_id: str,
1202 ) -> ContentItem | None:
1203     """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
1204     conn.row_factory = aiosqlite.Row
1205     async with conn.execute(
1206         """SELECT * FROM content_items
1207             WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1208         (account_id, source_id, provider_message_id),
1209     ) as cur:
1210         row = await cur.fetchone()
1211         return _row_to_content_item(row) if row else None
1212
1213
1214 async def get_content_item_by_dedup_key(
1215     conn: aiosqlite.Connection, account_id: int, dedup_key: str
1216 ) -> ContentItem | None:
1217     """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1218
1219     Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1220     a single query: the ``dedup_key`` already encodes whichever tier was
1221     applicable at ingestion time.
1222     """
1223     conn.row_factory = aiosqlite.Row
1224     async with conn.execute(
1225         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1226         (account_id, dedup_key),

```

```

1227         ) as cur:
1228             row = await cur.fetchone()
1229             return _row_to_content_item(row) if row else None
1230
1231
1232 # --- Policy repos (ADR 0011, Option 2A) -----
1233
1234
1235 async def upsert_policy(conn: aiomysqlite.Connection, policy: Policy) -> Policy:
1236     """Insert or update a policy row by ``(account_id, name)``.
1237
1238     Exactly one policy per account should carry ``is_default=True``; callers
1239     are responsible for flipping the previous default before flipping a new
1240     one (enforced by a partial unique index landed by future work, not in the
1241     schema ? SQLite ``CHECK`` can't express "at most one per group").
1242     """
1243     await conn.execute(
1244         """INSERT INTO policies (account_id, name, config_json, is_default,
1245                                created_at, updated_at)
1246         VALUES (?, ?, ?, ?, ?, ?)
1247         ON CONFLICT(account_id, name) DO UPDATE SET
1248             config_json=excluded.config_json,
1249             is_default=excluded.is_default,
1250             updated_at=excluded.updated_at""",
1251         (
1252             policy.account_id,
1253             policy.name,
1254             policy.config_json,
1255             int(policy.is_default),
1256             policy.created_at.isoformat(),
1257             policy.updated_at.isoformat(),
1258         ),
1259     )
1260     await conn.commit()
1261     conn.row_factory = aiomysqlite.Row
1262     async with conn.execute(
1263         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1264         (policy.account_id, policy.name),
1265     ) as cur:
1266         row = await cur.fetchone()
1267         assert row is not None
1268         return _row_to_policy(row)
1269
1270
1271 async def get_default_policy(conn: aiomysqlite.Connection, account_id: int) -> Policy |
None:
1272     """Return the account's default policy (``is_default=1``)."""
1273     conn.row_factory = aiomysqlite.Row
1274     async with conn.execute(
1275         "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1276         (account_id, ),
1277     ) as cur:
1278         row = await cur.fetchone()
1279         return _row_to_policy(row) if row else None
1280
1281
1282 # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1283
1284
1285 async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1286     """Insert or update a destination row by (account, provider, target)."""
1287     await conn.execute(
1288         """INSERT INTO destinations
1289         (account_id, provider, target, kind, title, is_output, created_at)

```

```

1290         VALUES (?, ?, ?, ?, ?, ?, ?)
1291         ON CONFLICT(account_id, provider, target) DO UPDATE SET
1292             kind=excluded.kind,
1293             title=excluded.title,
1294             is_output=excluded.is_output"",
1295         (
1296             dest.account_id,
1297             dest.provider,
1298             dest.target,
1299             dest.kind,
1300             dest.title,
1301             int(dest.is_output),
1302             dest.created_at.isoformat(),
1303         ),
1304     )
1305     await conn.commit()
1306     conn.row_factory = aiosqlite.Row
1307     async with conn.execute(
1308         """SELECT * FROM destinations
1309             WHERE account_id = ? AND provider = ? AND target = ?""",
1310         (dest.account_id, dest.provider, dest.target),
1311     ) as cur:
1312         row = await cur.fetchone()
1313     assert row is not None
1314     return _row_to_destination(row)
1315
1316
1317 async def get_destination(
1318     conn: aiosqlite.Connection, account_id: int, provider: str, target: str
1319 ) -> Destination | None:
1320     """Resolve a destination by its (account, provider, target) natural key."""
1321     conn.row_factory = aiosqlite.Row
1322     async with conn.execute(
1323         "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target = ?"
1324         ,
1325         (account_id, provider, target),
1326     ) as cur:
1327         row = await cur.fetchone()
1328     return _row_to_destination(row) if row else None
1329
1330 async def record_delivery(
1331     conn: aiosqlite.Connection,
1332     *,
1333     content_item_id: int,
1334     destination_id: int,
1335     route_id: int | None = None,
1336     success: bool,
1337     error: str | None = None,
1338 ) -> None:
1339     """Record a delivery outcome for one (content_item, destination) pair.
1340
1341     This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
1342     sets status=delivered; on failure, increments attempt_count and records the
1343     error. Idempotent: repeated calls update the same row.
1344     """
1345     status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
1346     await conn.execute(
1347         """INSERT INTO delivery_attempts
1348             (content_item_id, destination_id, route_id, status, attempt_count,
1349             last_error, created_at, delivered_at)
1350         VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
1351             CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
1352         ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
1353             status=excluded.status,

```

```

1354         attempt_count=delivery_attempts.attempt_count + 1,
1355         last_error=excluded.last_error,
1356         delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
1357                     ELSE delivery_attempts.delivered_at END""",
1358         (
1359             content_item_id,
1360             destination_id,
1361             route_id,
1362             status.value,
1363             error,
1364             success,
1365             success,
1366         ),
1367     )
1368     await conn.commit()
1369
1370
1371     async def get_delivery_attempt(
1372         conn: aiomysqlite.Connection, content_item_id: int, destination_id: int
1373     ) -> DeliveryAttempt | None:
1374         """Read a delivery attempt row by its natural key (idempotency boundary)."""
1375         conn.row_factory = aiomysqlite.Row
1376         async with conn.execute(
1377             "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
1378             ?",
1379             (content_item_id, destination_id),
1380         ) as cur:
1381             row = await cur.fetchone()
1382             return _row_to_delivery_attempt(row) if row else None
1383
1384     # --- legacy Channel shim (backwards-compatible view over sources) -----
1385
1386
1387     async def upsert_channel(conn: aiomysqlite.Connection, channel: Channel) -> Channel:
1388         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1389

```

9. user (2026-07-06T09:07:26.550Z)

```

1     """Tests for the provider-independent CommandService (command_service.py).
2
3     These tests exercise command parsing and config mutation without any
4     Telethon objects ? pure strings in, CommandResult out. The Telethon
5     adapter (commands.py) is tested separately via test_commands.py.
6     """
7
8     from __future__ import annotations
9
10    import aiomysqlite
11    import pytest
12
13    from tg_video_filter import db
14    from tg_video_filter.command_service import (
15        CommandService,
16        format_commands,
17        parse_duration_s,
18        parse_size_mb,
19    )
20    from tg_video_filter.models import FilterConfig, Source
21
22    USER_ID = 111
23
24
25    @pytest.fixture

```

```

26     async def conn() -> aiosqlite.Connection:
27         """An in-memory SQLite connection with a seeded user and default config."""
28         c = await aiosqlite.connect(":memory:")
29         await db._apply_schema(c)
30         await db.upsert_user(
31             c,
32             db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33         )
34         await db.save_filter_config(c, USER_ID, FilterConfig())
35         yield c
36         await c.close()
37
38
39     @pytest.fixture
40     def svc() -> CommandService:
41         return CommandService()
42
43
44     # --- unit parser tests -----
45
46
47     @pytest.mark.parametrize(
48         "value, unit, expected",
49         [
50             ("10", "MB", 10),
51             ("1", "GB", 1024),
52             ("500", "KB", 1),
53             ("5", "", 5),
54             ("2", "TB", 2 * 1024 * 1024),
55         ],
56     )
57     def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
58         assert parse_size_mb(value, unit) == expected
59
60
61     def test_parse_size_mb_unknown_unit_raises() -> None:
62         with pytest.raises(ValueError):
63             parse_size_mb("10", "XB")
64
65
66     @pytest.mark.parametrize(
67         "value, unit, expected",
68         [
69             ("15", "m", 900),
70             ("2", "h", 7200),
71             ("90", "s", 90),
72             ("5", "", 5),
73         ],
74     )
75     def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
76         assert parse_duration_s(value, unit) == expected
77
78
79     def test_parse_duration_s_unknown_unit_raises() -> None:
80         with pytest.raises(ValueError):
81             parse_duration_s("10", "x")
82
83
84     # --- /set size -----
85
86
87     async def test_set_size(conn: aiosqlite.Connection, svc: CommandService) -> None:
88         result = await svc.handle(USER_ID, "/set size 100MB", conn, FilterConfig())
89         assert "100MB" in result.reply_text
90         assert result.config_changed is True

```

```

91         loaded = await db.load_filter_config(conn, USER_ID)
92         assert loaded.max_size_mb == 100
93
94
95     async def test_set_size_regex_no_match_returns_empty(
96         conn: aiomysql.Connection, svc: CommandService
97     ) -> None:
98         """Input that doesn't match the _SIZE_RE regex returns empty (noise path).
99
100         The regex enforces valid units ? this isn't testing ValueError from
101         parse_size_mb (that branch is unreachable). See review finding #3.
102         """
103         result = await svc.handle(USER_ID, "/set size not-a-number", conn, FilterConfig())
104         assert result.reply_text == ""
105         assert result.config_changed is False
106
107
108     async def test_set_size_below_minimum(conn: aiomysql.Connection, svc: CommandService)
-> None:
109         result = await svc.handle(USER_ID, "/set size 0MB", conn, FilterConfig())
110         assert "at least 1 MB" in result.reply_text
111         assert result.config_changed is False
112
113
114     # --- /set length -----
115
116
117     async def test_set_length(conn: aiomysql.Connection, svc: CommandService) -> None:
118         result = await svc.handle(USER_ID, "/set length 15m", conn, FilterConfig())
119         assert "15m" in result.reply_text or "900s" in result.reply_text
120         assert result.config_changed is True
121         loaded = await db.load_filter_config(conn, USER_ID)
122         assert loaded.max_duration_s == 900
123
124
125     async def test_set_length_below_min(conn: aiomysql.Connection, svc: CommandService) ->
None:
126         cfg = FilterConfig(min_duration_s=60)
127         result = await svc.handle(USER_ID, "/set length 30s", conn, cfg)
128         assert "must be >= min duration" in result.reply_text
129         assert result.config_changed is False
130
131
132     # --- /set min length -----
133
134
135     async def test_set_min_length(conn: aiomysql.Connection, svc: CommandService) -> None:
136         result = await svc.handle(USER_ID, "/set min length 5s", conn, FilterConfig())
137         assert "5s" in result.reply_text
138         assert result.config_changed is True
139         loaded = await db.load_filter_config(conn, USER_ID)
140         assert loaded.min_duration_s == 5
141
142
143     async def test_set_min_length_above_max(conn: aiomysql.Connection, svc: CommandService)
-> None:
144         cfg = FilterConfig(max_duration_s=10)
145         result = await svc.handle(USER_ID, "/set min length 60s", conn, cfg)
146         assert "must be <= max duration" in result.reply_text
147         assert result.config_changed is False
148
149
150     # --- /set min resolution -----
151
152

```

```

153     async def test_set_min_resolution(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
154         result = await svc.handle(USER_ID, "/set min resolution 720p", conn, FilterConfig())
155         assert "720p" in result.reply_text
156         assert result.config_changed is True
157         loaded = await db.load_filter_config(conn, USER_ID)
158         assert loaded.target_height == 720
159
160
161     async def test_set_min_resolution_below_min(
162         conn: aiomysqlite.Connection, svc: CommandService
163     ) -> None:
164         result = await svc.handle(USER_ID, "/set min resolution 0p", conn, FilterConfig())
165         assert "at least 1p" in result.reply_text
166         assert result.config_changed is False
167
168
169     async def test_set_min_res_help_on_malformed(
170         conn: aiomysqlite.Connection, svc: CommandService
171     ) -> None:
172         result = await svc.handle(USER_ID, "/set min resolution abc", conn, FilterConfig())
173         assert "Usage:" in result.reply_text
174
175
176     # --- /set exclude -----
177
178
179     async def test_set_exclude_gif_on(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
180         result = await svc.handle(USER_ID, "/set exclude gif on", conn, FilterConfig())
181         assert "ON" in result.reply_text
182         assert result.config_changed is True
183         loaded = await db.load_filter_config(conn, USER_ID)
184         assert loaded.exclude_gif is True # verify correct field mutated
185
186
187     async def test_set_exclude_round_off(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
188         result = await svc.handle(USER_ID, "/set exclude round off", conn, FilterConfig())
189         assert "OFF" in result.reply_text
190         assert result.config_changed is True
191
192
193     async def test_set_exclude_help_on_malformed(
194         conn: aiomysqlite.Connection, svc: CommandService
195     ) -> None:
196         result = await svc.handle(USER_ID, "/set exclude gif true", conn, FilterConfig())
197         assert "Usage:" in result.reply_text
198         assert result.config_changed is False
199
200
201     # --- /show config -----
202
203
204     async def test_show_config(conn: aiomysqlite.Connection, svc: CommandService) -> None:
205         result = await svc.handle(USER_ID, "/show config", conn, FilterConfig())
206         assert "Current Filter Configuration" in result.reply_text
207         assert result.config_changed is False
208
209
210     # --- /show commands -----
211
212
213     async def test_show_commands(conn: aiomysqlite.Connection, svc: CommandService) -> None:
214         result = await svc.handle(USER_ID, "/show commands", conn, FilterConfig())

```

```

215         assert "Available Commands" in result.reply_text
216         assert result.config_changed is False
217
218
219     # --- unrecognized command -----
220
221
222     async def test_unrecognized_returns_empty(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
223         result = await svc.handle(USER_ID, "hello world", conn, FilterConfig())
224         assert result.reply_text == ""
225         assert result.config_changed is False
226
227
228     # --- format_commands -----
229
230
231     def test_format_commands_includes_all_sections() -> None:
232         text = format_commands()
233         assert "/show config" in text
234         assert "/set size" in text
235         assert "/set exclude" in text
236         assert "/backfill" in text
237
238
239     # --- /channels + /watch + /unwatch + /watchlist (ADR 0009, Track E) -----
240
241
242     async def _seed_source(
243         conn: aiomysqlite.Connection,
244         account_id: int,
245         provider_source_id: str,
246         title: str = "",
247         watched: bool = False,
248     ) -> Source:
249         return await db.upsert_source(
250             conn,
251             Source(
252                 account_id=account_id,
253                 provider="telegram",
254                 provider_source_id=provider_source_id,
255                 title=title,
256                 kind="channel",
257                 watched=watched,
258             ),
259         )
260
261
262     async def test_channels_lists_all_sources_with_watch_status(
263         conn: aiomysqlite.Connection, svc: CommandService
264     ) -> None:
265         await _seed_source(conn, USER_ID, "-1001", "Channel A", watched=True)
266         await _seed_source(conn, USER_ID, "-1002", "Channel B", watched=False)
267         result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
268         assert "Channel A" in result.reply_text
269         assert "Channel B" in result.reply_text
270         assert "?" in result.reply_text # watched indicator
271         assert "?" in result.reply_text # unwatched indicator
272
273
274     async def test_channels_empty_shows_help(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
275         result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
276         assert "No sources yet" in result.reply_text
277

```

```

278
279     async def test_watchlist_only_watched(conn: aiosqlite.Connection, svc: CommandService)
-> None:
280         await _seed_source(conn, USER_ID, "-1001", "Ch A", watched=True)
281         await _seed_source(conn, USER_ID, "-1002", "Ch B", watched=False)
282         result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
283         assert "Ch A" in result.reply_text
284         assert "Ch B" not in result.reply_text
285
286
287     async def test_watchlist_empty_shows_help(conn: aiosqlite.Connection, svc:
CommandService) -> None:
288         result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
289         assert "No watched sources" in result.reply_text
290
291
292     async def test_watch_by_numeric_id(conn: aiosqlite.Connection, svc: CommandService) ->
None:
293         await _seed_source(conn, USER_ID, "-1001", "Target", watched=False)
294         result = await svc.handle(USER_ID, "/watch -1001", conn, FilterConfig())
295         assert "Now watching" in result.reply_text
296         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
297         assert src is not None
298         assert src.watched is True
299
300
301     async def test_watch_by_username(conn: aiosqlite.Connection, svc: CommandService) ->
None:
302         await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
303         result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
304         assert "Now watching" in result.reply_text
305         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
306         assert src is not None
307         assert src.watched is True
308
309
310     async def test_unwatch(conn: aiosqlite.Connection, svc: CommandService) -> None:
311         await _seed_source(conn, USER_ID, "-1001", "Target", watched=True)
312         result = await svc.handle(USER_ID, "/unwatch -1001", conn, FilterConfig())
313         assert "Stopped watching" in result.reply_text
314         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
315         assert src is not None
316         assert src.watched is False
317
318
319     async def test_watch_unknown_source_shows_error(
320         conn: aiosqlite.Connection, svc: CommandService
321     ) -> None:
322         result = await svc.handle(USER_ID, "/watch -9999", conn, FilterConfig())
323         assert "not found" in result.reply_text
324
325
326     async def test_set_source_watched_toggle(conn: aiosqlite.Connection) -> None:
327         src = await _seed_source(conn, USER_ID, "-2000", watched=True)
328         assert src.id is not None
329         await db.set_source_watched(conn, USER_ID, src.id, False)
330         reloaded = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-2000")
331         assert reloaded is not None
332         assert reloaded.watched is False
333
334
335     async def test_list_account_sources_watched_only(
336         conn: aiosqlite.Connection,
337     ) -> None:
338         await _seed_source(conn, USER_ID, "-10", watched=True)

```

```

339         await _seed_source(conn, USER_ID, "-20", watched=False)
340     all_srcs = await db.list_account_sources(conn, USER_ID)
341     assert len(all_srcs) == 2
342     watched = await db.list_account_sources(conn, USER_ID, watched_only=True)
343     assert len(watched) == 1
344     assert watched[0].provider_source_id == "-10"
345

```

10. assistant (2026-07-06T09:07:29.220Z)

Now let me look at the two new db functions (`set\_source\_watched`, `list\_account\_sources`) and the commands.py re-export change.

11. user (2026-07-06T09:07:30.909Z)

```

1537:async def set_source_watched(
1538-     conn: aiomysqlite.Connection, account_id: int, source_id: int, watched: bool
1539-) -> None:
1540-     """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
1541-     await conn.execute(
1542-         "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
1543-         (int(watched), source_id, account_id),
1544-     )
1545-     await conn.commit()
1546-
1547-
1548:async def list_account_sources(
1549-     conn: aiomysqlite.Connection, account_id: int, *, watched_only: bool = False
1550-) -> list[Source]:
1551-     """List all sources for an account, optionally filtering to watched only."""
1552-     conn.row_factory = aiomysqlite.Row
1553-     query = "SELECT * FROM sources WHERE account_id = ?"
1554-     params: tuple = (account_id,)
1555-     if watched_only:
1556-         query += " AND watched = 1"
1557-     query += " ORDER BY watched DESC, provider_source_id ASC"
1558-     async with conn.execute(query, params) as cur:
1559-         rows = await cur.fetchall()
1560-     return [_row_to_source(r) for r in rows]
1561-
1562-
1563:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1564-     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1565-
1566-     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1567-     if absent), computes the provider-neutral ``dedup_key`` via
1568-     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1569-     ``VideoRecord`` shape is reconstructed from the inserted row.
1570-     """
1571-     src = await upsert_source(
1572-         conn,
1573-         Source(
1574-             account_id=video.user_id,
1575-             provider="telegram",
1576-             provider_source_id=str(video.channel_id),
1577-             kind="channel",
1578-             watched=True,
1579-         ),
1580-     )
1581-     # ...
1582-
1583-
1584-# Commands
1585-commands = {
1586-     "set_source_watched",
1587-     "list_account_sources",
1588-     "insert_content_item",
1589-     "get_content_item_by_message",
1590-     "get_content_item_by_dedup_key",
1591-     "upsert_policy",
1592-     "get_default_policy",
1593- }

```

```

1753- # Destination / Routing / Delivery repos (ADR 0011, Track C)
1754- "upsert_destination",
1755- "get_destination",
1756- "record_delivery",
1757- "get_delivery_attempt",
1758- # Legacy User shims (backwards-compatible)
1759- "upsert_user",
1760- "get_user",
1761- "list_enabled_users",
1762- # Legacy Channel / Video / config shims (ADR 0011)
1763- "upsert_channel",
1764- "get_channel",
1765- "get_video",
1766- "get_video_by_document_id",
1767- "get_video_by_content_fingerprint",
1768- "insert_video",
1769- "mark_forwarded",
1770- "load_filter_config",
1771- "save_filter_config",
1772- "list_videos_for_user",
1773-]
1774-
1775-
1776-def __dir__() -> Iterable[str]:
1777-    return __all__

```

12. user (2026-07-06T09:07:32.327Z)

```

diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 974be99..3aea1f6 100644
--- a/src/tg_video_filter/commands.py
+++ b/src/tg_video_filter/commands.py
@@ -27,7 +27,8 @@ from tg_video_filter.models import FilterConfig, User

log = logging.getLogger(__name__)

-from tg_video_filter.command_service import ( # noqa: E402
+from tg_video_filter.command_service import ( # noqa: E402, F401
+    _CHANNELS_RE,
+    _EXCLUDE_HELP_RE,
+    _EXCLUDE_RE,
+    _LENGTH_RE,
@@ -37,6 +38,9 @@ from tg_video_filter.command_service import ( # noqa: E402
     _SHOW_COMMANDS_RE,
     _SHOW_CONFIG_RE,
     _SIZE_RE,
+    _UNWATCH_RE,
+    _WATCH_RE,
+    _WATCHLIST_RE,
     _format_duration,
     _format_size,
     format_commands,

```

13. assistant (2026-07-06T09:07:37.103Z)

Now let me check the key questions. First, the `upsert\_channel` shim (the lazy-population path) to verify what `watched` value it sets for newly discovered channels, and whether `on\_new\_message`'s gate lets an unwatched-but-existing channel through. Let me read the `upsert\_channel` shim.

14. user (2026-07-06T09:07:37.808Z)

```

1387 async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1388     """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.

```

```

1389
1390     ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1391     reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1392     New code should call :func:`upsert_source` directly.
1393     """
1394     src = await upsert_source(
1395         conn,
1396         Source(
1397             account_id=channel.user_id,
1398             provider="telegram",
1399             provider_source_id=str(channel.channel_id),
1400             title=channel.title,
1401             kind="channel",
1402             watched=True, # back-compat: pre-ADR-0011 channels were all watched
1403             seen_first_at=channel.seen_first_at,
1404         ),
1405     )
1406     return _source_to_channel(src)
1407
1408
1409     async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1410         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1411         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1412         return _source_to_channel(src) if src else None
1413
1414
1415     def _source_to_channel(src: Source) -> Channel:
1416         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.
1417
1418         Only valid for Telegram sources (``provider == "telegram"``); the legacy
1419         ``Channel.channel_id`` is a Telegram-specific int id. Non-Telegram sources
1420         raise ``ValueError`` ? new code should use :class:`Source` directly instead
1421         of the deprecated :class:`Channel` shim (ADR 0011 provider-neutrality guard).
1422         """
1423         if src.provider != "telegram":
1424             raise ValueError(
1425                 f"Cannot project {src.provider!r} source onto the Telegram-specific "
1426                 f"Channel type. Use Source directly (ADR 0011)."
1427             )
1428         return Channel(
1429             id=src.id,
1430             user_id=src.account_id,
1431             channel_id=int(src.provider_source_id),
1432             title=src.title,
1433             seen_first_at=src.seen_first_at,
1434         )
1435
1436
1437     # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1438
1439
1440     async def get_video(
1441         conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
1442     ) -> VideoRecord | None:
1443         """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1444
1445         Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1446         :class:`Source` + :class:`ContentItem` and projects the latter onto a
1447         :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1448         """
1449         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1450         if src is None:
1451             return None
1452         item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))

```

```

1453         return _content_item_to_video(item, src, user_id, channel_id) if item else None
1454
1455
1456     async def get_video_by_document_id(
1457         conn:aiosqlite.Connection, user_id: int, document_id: int
1458     ) -> VideoRecord | None:
1459         """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1460         dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1461         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1462         if item is None:
1463             return None
1464         src = await get_source_by_id(conn, item.source_id)
1465         if src is None:
1466             return None

```

15. assistant (2026-07-06T09:07:44.700Z)

This confirms a critical ADR violation. Now I have the full picture. Let me trace the exact behavior of the gate against ADR 0009 line 130-131.

The gate in `telethon\_client.py:189-191`:

```

```python
src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
if src is not None and not src.watched:
 return # silently skip unwatched source
```

```

When a NEW channel (no source row) sends its first message: `src is None`, so the gate does NOT return ? it falls through to dedup, then at line 249 calls `db.upsert\_channel(...)` which the shim (db.py:1402) hard-codes `watched=True`. So a brand-new channel is ingested AND auto-watched. This directly violates ADR 0009 line 130-131 ("New channels discovered after the migration default to watched = 0 and must be explicitly watched") and line 37-38 (DEFAULT 0 opt-in). Let me verify this with a repro to be certain, and also check `/channels` for the `get\_dialogs()` upsert requirement.

16. user (2026-07-06T09:07:50.852Z)

NEW channel via lazy upsert_channel -> watched = True

17. user (2026-07-06T09:07:50.978Z)

Found 4 files
docs\adr\0009-selective-channel-scanning.md
src\tg_video_filter\backfill.py
scripts\resolve_target.py
docs\runbook.md

18. assistant (2026-07-06T09:07:59.652Z)

Confirmed the auto-watch bug. `get\_dialogs` is not referenced in `command\_service.py` ? so `/channels` does NOT upsert candidate rows from `client.get\_dialogs()`. Let me verify the `/channels` handler has no dialogs access (it can't ? it takes only `conn, user\_id`, no client), confirming ADR 0009 lines 48-54 discoverability requirement is unmet. Let me also confirm the gate's full flow: an unwatched existing source correctly returns, but does the "new channel" path proceed to full ingestion + delivery.

Let me verify one more nuance ? after `upsert\_channel` sets watched=True on first sight, the message that triggered it is ALSO fully processed and delivered in that same call (not just recorded). Reading the flow: gate passes (src is None) ? dedup ? upsert_channel(watched=True) ? evaluate ? insert ? deliver. Yes, the very first message from any new channel is ingested and delivered. This is the "scan everything" behavior ADR 0009 explicitly set out to eliminate.

Now let me confirm the `format\_commands()` gap and check whether the `/watch` resolve-by-name works (ADR line 64 says resolve by name OR id, but sources only store numeric provider_source_id

for real Telegram channels).

19. user (2026-07-06T09:08:03.484Z)

```
'/watch' in /show commands help: False
'/unwatch' in /show commands help: False
'/channels' in /show commands help: False
'/watchlist' in /show commands help: False
```

20. assistant (2026-07-06T09:08:14.091Z)

Confirmed: none of the four new commands appear in ``/show commands``. Now let me verify the ``_resolve_source`` by-name concern. ADR line 64 says ``/watch <#name|id>``. For real Telegram sources ``provider_source_id`` is the numeric channel id (per the task note and ``upsert_channel`` using ``str(channel_id)``). The ``_resolve_source`` only queries ``provider_source_id``, so a user typing ``/watch @channelname`` will fail for real channels since the stored id is numeric. Let me confirm real sources never store ``@username`` by checking how sources get created.

21. user (2026-07-06T09:08:15.518Z)

```
src\tg_video_filter\db.py:848:         provider_source_id=row["provider_source_id"],
src\tg_video_filter\db.py:1399:         provider_source_id=str(channel.channel_id),
src\tg_video_filter\db.py:1576:         provider_source_id=str(video.channel_id),
```

22. assistant (2026-07-06T09:08:22.635Z)

Confirmed. Every code path that creates a real Telegram source stores ``provider_source_id = str(channel_id)`` ? a numeric string like ``-1001234567890``. There is no path that stores an ``@username``. So ``_resolve_source`` can only ever match by numeric id in practice. The ``/watch @mychan`` test (test line 301) passes only because the test manually seeds a source with ``provider_source_id="@mychan"``, which never happens in production. ADR line 64's "resolve by #name/username OR id" is therefore effectively unimplemented for real sources ? resolving by channel name/username always fails.

Let me double-check the gate handles the unwatched-existing-source case correctly (it does return), and confirm there's genuinely no test exercising the gate logic. Let me search the whole test suite for the gate.

23. user (2026-07-06T09:08:23.750Z)

No matches found

24. assistant (2026-07-06T09:08:32.062Z)

Confirmed: no test touches the gate at all. The gate logic (``src`` is not `None` and not `src.watched``) lives entirely inside the ``# pragma: no cover`` ``on_new_message``, so the 97.10% coverage number excludes it. The gate is untestable as written because it's inlined into the Telethon handler rather than extracted into a pure helper.

I have enough to report. Let me compile the findings, ranked most-severe first.

Key findings:

1. ****Blocker/High****: New channel auto-watch violates ADR opt-in default ? the gate lets ``src`` is `None`` fall through, then ``upsert_channel`` hard-codes ``watched=True``, so every newly-discovered channel is scanned AND delivered on first message. Directly contradicts ADR 0009 lines 37-38 and 130-131. This defeats the feature's entire purpose.
2. ****High****: Gate is inlined in a ``# pragma: no cover`` handler with zero tests ? the exact bug in #1 would have been caught by a test.
3. ****Medium****: ``/channels`` does not upsert candidates from ``client.get_dialogs()`` (ADR lines 48-54) ? undiscoverable channels can't be opted into; the handler has no client access.
4. ****Medium****: ``/watch <#name|id>`` can't resolve by name/username for real sources (ADR line 64) ? sources only ever store numeric ids.
5. ****Low****: ``format_commands()`` omits all four new commands ? undiscoverable via ``/show commands``.

25. user (2026-07-06T09:09:16.695Z)

Structured output provided successfully